Travlr

Project Software Design Document

Version 3.0

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	05/16/26	Creston Getz	Summary, Design Constraints, System Architecture View
2.0	06/03/26	Creston Getz	Architecture View, Class Diagram, Interfaces
3.0	06/16/26	Creston Getz	User Interface, API Endpoints

Executive Summary

The Travlr web application is designed for customers to book travel packages with the Travlr Getaways company. Customers will be able to create an account, search for packages with filters, and book reservations. Customers will also be able to use the website to track their itineraries and other information about their trip. There will also be an admin only site where Travlr Getaways can manage their customers, trip packages and the pricing for the trips.

The application will be split into two architectures. The Model, View, Controller or MVC pattern will be the appropriate architecture for the customer part of the application. It separates the logic of the application allowing for easier debugging, improved security, scalability, and code re-use. This web application will use the MEAN (MongoDB, Express, Node, and Angular) stack for development. The Angular framework is good at making single page applications (SPA) which is ideal for the admin page. A SPA is best for the admin page due to increased performance. The load time is less important for the admins as once they load it; they need it to be responsive. While the MVC pattern increased security and scalability for the customer facing side of the application.

Design Constraints

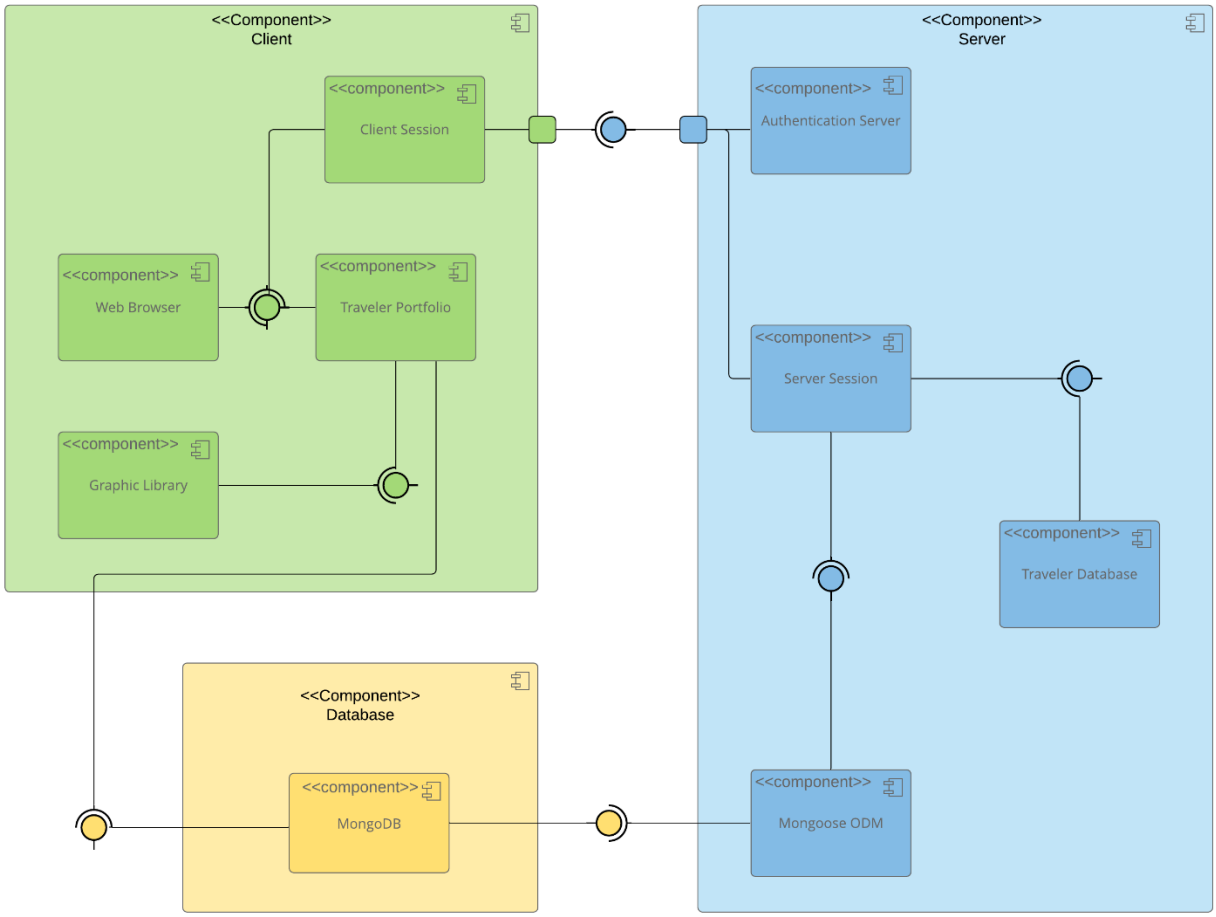
The Travlr application has a few design constraints. The use of the MEAN stack, the MVC pattern, and the SPA pattern for the web-based application all have implications of their own. While the MVC pattern offers many benefits, it increases the complexity of the application. There is a strict separation of logic between the model, view and controller parts of the application. Managing these files and structures will increase setup time and complexity. The single page application for the admin page will have increased testing time, slower initial load speeds, and potential problems with managing the saved data. The benefits of using the MVC pattern outweigh these constraints. And using a SPA for the admin page will give it the performance it needs.

How the web application will be hosted is another concern. A cloud provider is recommended over Travlr Getaways managing their own infrastructure. The ideal provider will be chosen later to help prevent vendor lock-in. Using a cloud provider will help with scalability ensuring rapid growth will not slow the application down.

The Travlr application must work across all web browsers and screen sizes, mobile and desktop, to ensure seamless use. Speed is very important for web applications, and the most popular browsers and platforms should be heavily tested. The application will also need to adhere to the Web Content Accessibility Guidelines and security measures to adhere with regulations. Not adhering to these guidelines and regulations can warrant legal action and reduce the usability of the application. Other constraints such as budget, scope, and timeline are to be discussed with Travlr Getaways.

System Architecture View

Component Diagram

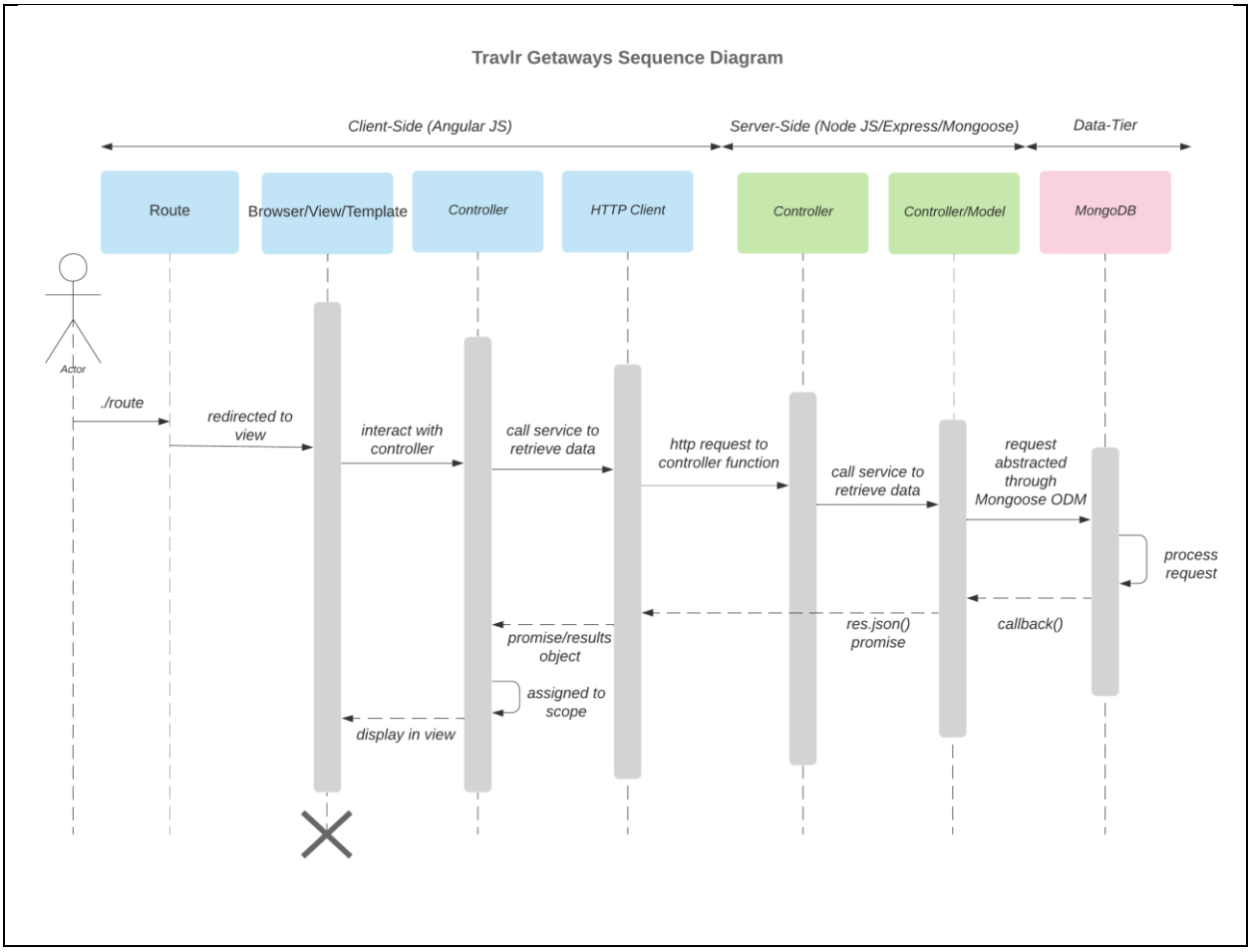


The component diagram above shows the MVC pattern for the customer part of the web application. The green section highlights the client or view (what the customer sees), the blue section highlights the server or backend of the application, and the yellow highlights the database of the application. Each of these is their own component. Starting with the database (MongoDB) of the application, there are two inputs (interfaces): one to the client and one to the server.

Moving into the server, which is connected to the database using a tool called Mongoose, it will send the data we need from the database to the server session which holds the business logic and the Travlr database. The only interaction between the server and the client is authenticating the user session, which is seen with the interface at the top. The server and client are connected via ports (square), which specify a separate interaction point between components and their environments. Put simply, the port defines how the parts are connected.

Inside of the client section, the Travlr portfolio of trips and packages is received from the database and the needed graphics (pictures, etc.) from the graphics library. Connecting the Travlr portfolio with a web browser allows the user to visit the URL, which will send a request to our sever to authenticate the user and send the needed files. Once authenticated the files are loaded onto the users browser showing the web page. This process will repeat when the user visits a new page on the site or needs data from the database. The user should stay authenticated by maintaining an authenticated session with the server via session cookies/tokens. These identify each user that is connected to the server and is how our systems keeps track of who is who.

Sequence Diagram



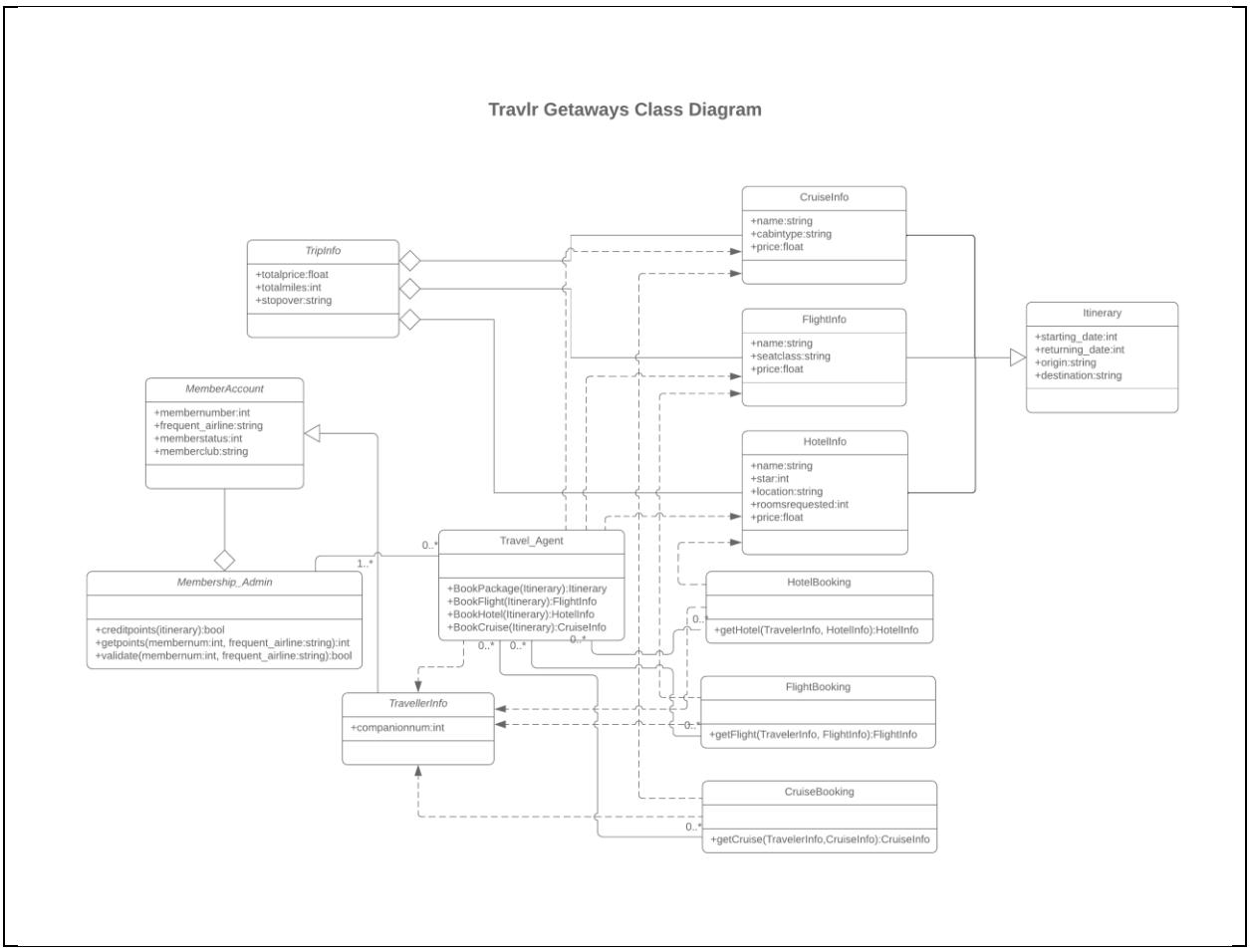
The flow of the web application can be shown in this sequence diagram. A sequence diagram is a way to show the order of operation/actions an actor takes, which is a user in this case. The diagram starts with the actor or stick figure on the left and walks through the steps the application will take going from left to right. The first four events are on the front end of our application or the client side. HTTP is a set of rules that allows web browsers and applications to talk to each other.

The user first visits the site either by clicking a link or typing in a URL; their request will be handled by a route, so they are sent to the correct location. The route is used to direct the user's web browser to the view which will show the HTML or the contents of the website to the user. For example, if a user clicks on a URL that takes them to the homepage, the route is how the application knows to send them to the home page and not a login screen or vice versa. Lastly, a controller will use an HTTP client to retrieve data from the server side of the application. This is done automatically as the contents of our web page are stored inside the database.

Once the server side or the back end of the application has received the HTTP request, it is handled by a controller and a model. The controller will take the HTTP request and act as a mediator before it talks to the database. The model is what talks directly to our Mongo database.

Once the request is handled by the database, which could be a user signing in or the data to fill out our webpage, it will be sent back to the model first which will then pass it to the controller. The controller will send it back to the client side of the application in the form of an HTTP response. The controller on the client side of the application will process the response and update the view to be shown on the actor's web browser. This process is done automatically upon loading a page. If a user is logging in, then they have to perform the actions to initiate the process. But this process is usually done at a speed the user does not notice.

Class Diagram



This picture above shows the class diagram for our application. A class diagram is a static representation of objects/classes in our application. For example, a travel agent is a person/object, and information about the hotel is an object. Class diagrams are very useful for showing relationships between objects. The arrows, lines, and diamonds describe these relationships in an efficient manner.

Each object or class is broken into three parts; notice how each square or rectangle has three sections inside of it. The top section is the name of the class, and the middle is the attributes of the class. An attribute is a way to describe something such as a person's name or the price of a flight. The bottom section is the operation the class can perform. A travel agent, for example, can book flights and

cruises. All of the attributes and methods in this diagram are public, which is what the + sign indicates. A public attribute/method means the other classes can see and use them.

Using this class diagram, we can efficiently see how each class is related. CruiseInfo, FlightInfo, and HotelInfo are all connected to Itinerary with a solid line and a hollow arrow indicating an "is-a" relationship. TripInfo is connected to CruiseInfo, FlightInfo, and HotelInfo with three solid lines and hollow diamonds. This is a "has-a" relationship. Trip info has information on a hotel, cruise, and flight. Membership_Admin also has a hollow diamond coming from MemberAccount. A member account has a member admin.

A Travel_Agent has many solid lines connecting to all of the booking classes: HotelBooking, FlightBooking, and TravellerInfo. A solid line indicates basic connections, and the little 0..* means one travel agent can manage many bookings or travelers. Likewise, the 1..* from Membership_Admin to Travel_Agent means that one or more admins will oversee zero or more travel agents.

The three booking classes have dashed lines pointing to their respective info classes. A dashed line is a "uses" relationship, so the booking classes will use the information inside the info classes. TravellerInfo has dashed lines coming to it from the booking classes and travel agent, which means they use the traveler's information. Without a traveler, there is no information on the trip or an agent to manage them. TravellerInfo also has a line with a hollow arrow (is-a relationship) to MemberAccount.

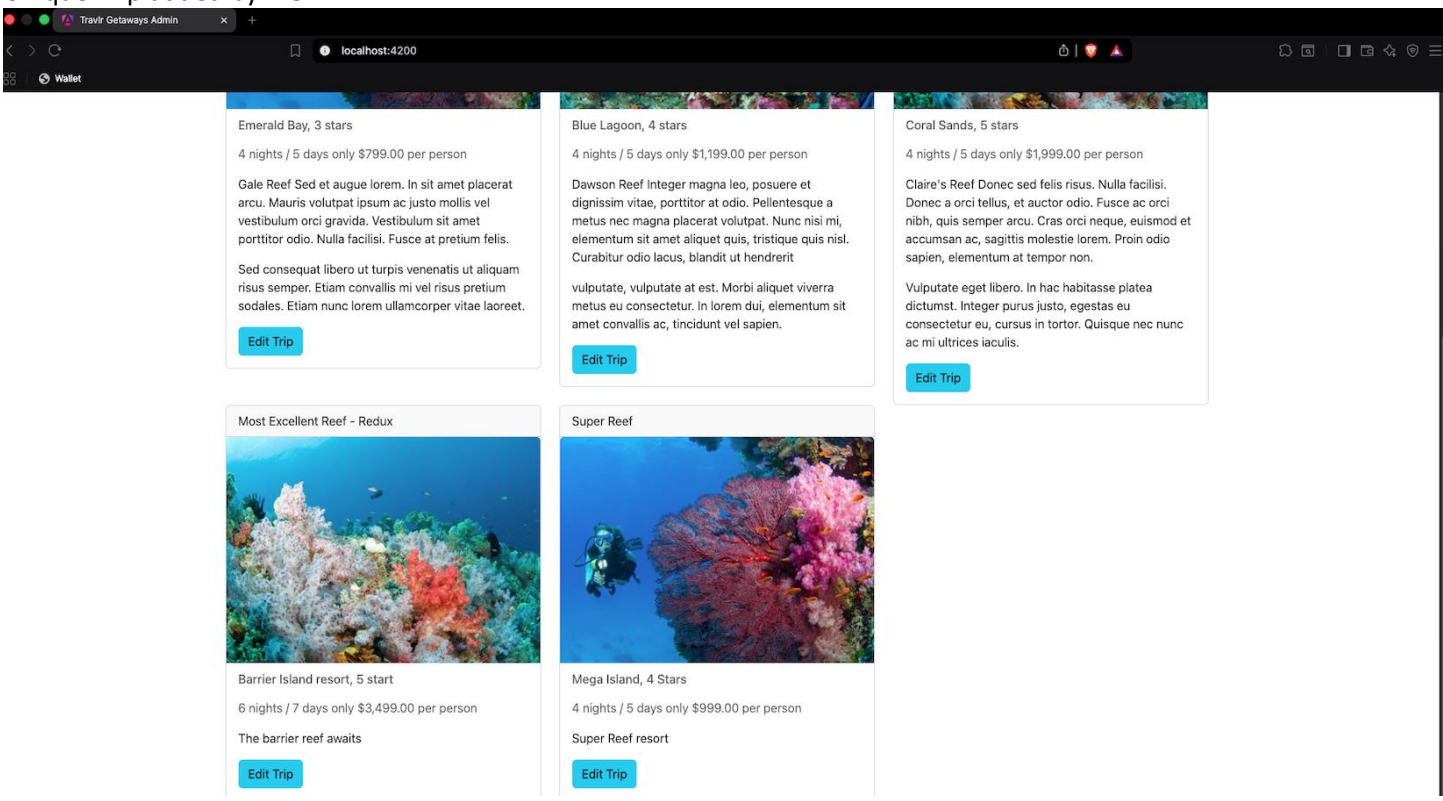
API Endpoints

Each of the endpoints sends a unique HTTP request. A GET request is used to retrieve data, a POST request is used to send new data, and a PUT method replaces or updates data that exists. These requests are what allow our applications to communicate with each other. A JSON web token or JWT is an encrypted token that allows the application to authenticate a user. When a user registers or logs in one is created telling the api that the user has access to the two restricted api’s.

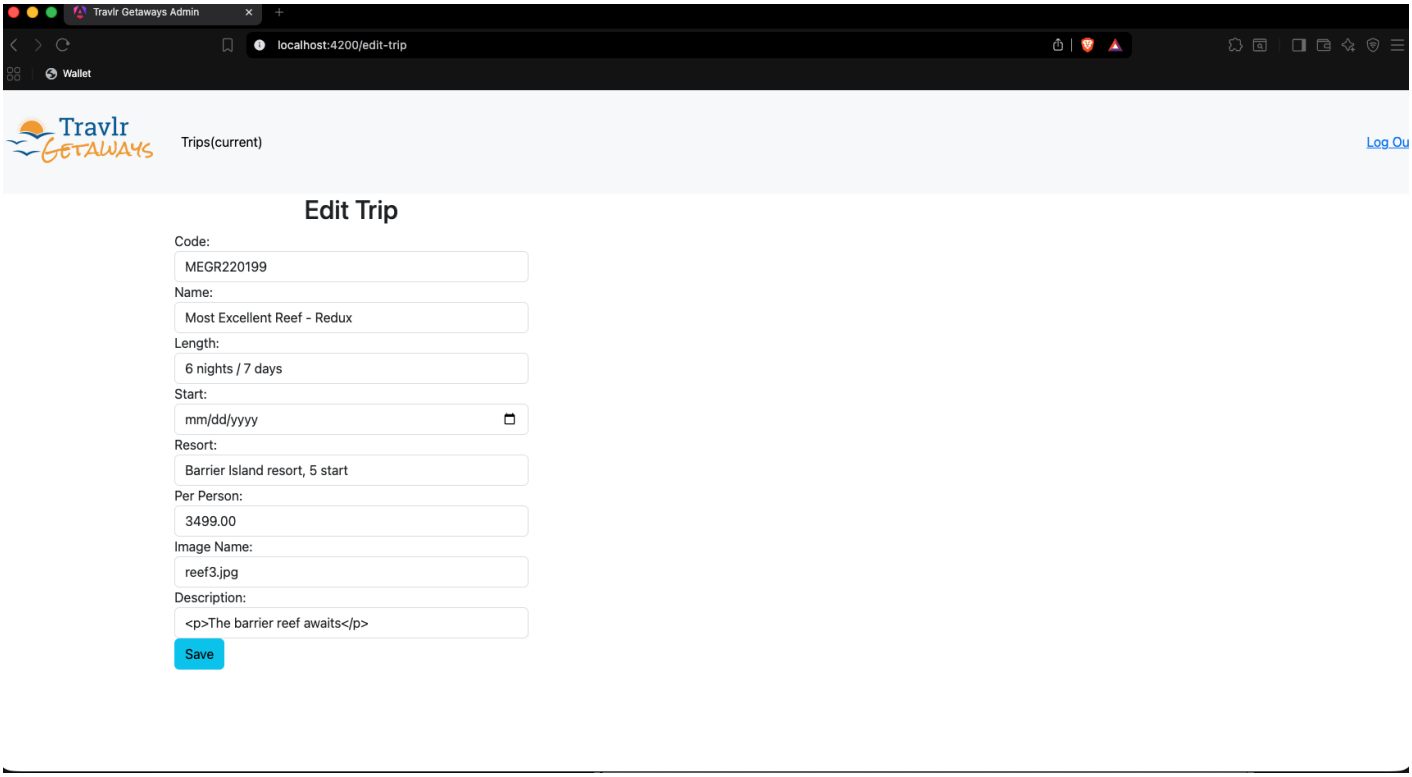
Method	Purpose	URL	Notes
GET	Get a list of all the trips	/api/trips	Returns a list of all Trips
GET	Filter the trips and only get one trip by its code	/api/trips/tripCode	Returns a single trip identified by the trip code inside the URL
POST	Adds a trip to the database	/api/trips	Send JSON to database. User must be logged in (JSON web token)
PUT	Edits the trips in the database.	/api/trips/:tripcode	Send JSON to Database. Requires user to be logged in(JSON web token)
POST	Registers a new user	/api/register	Creates a JSON web token on success. Hashes the password.
POST	Logs in an existing user	/api/login	Checked email and password hash and created a JSON web token upon valid login.

The User Interface

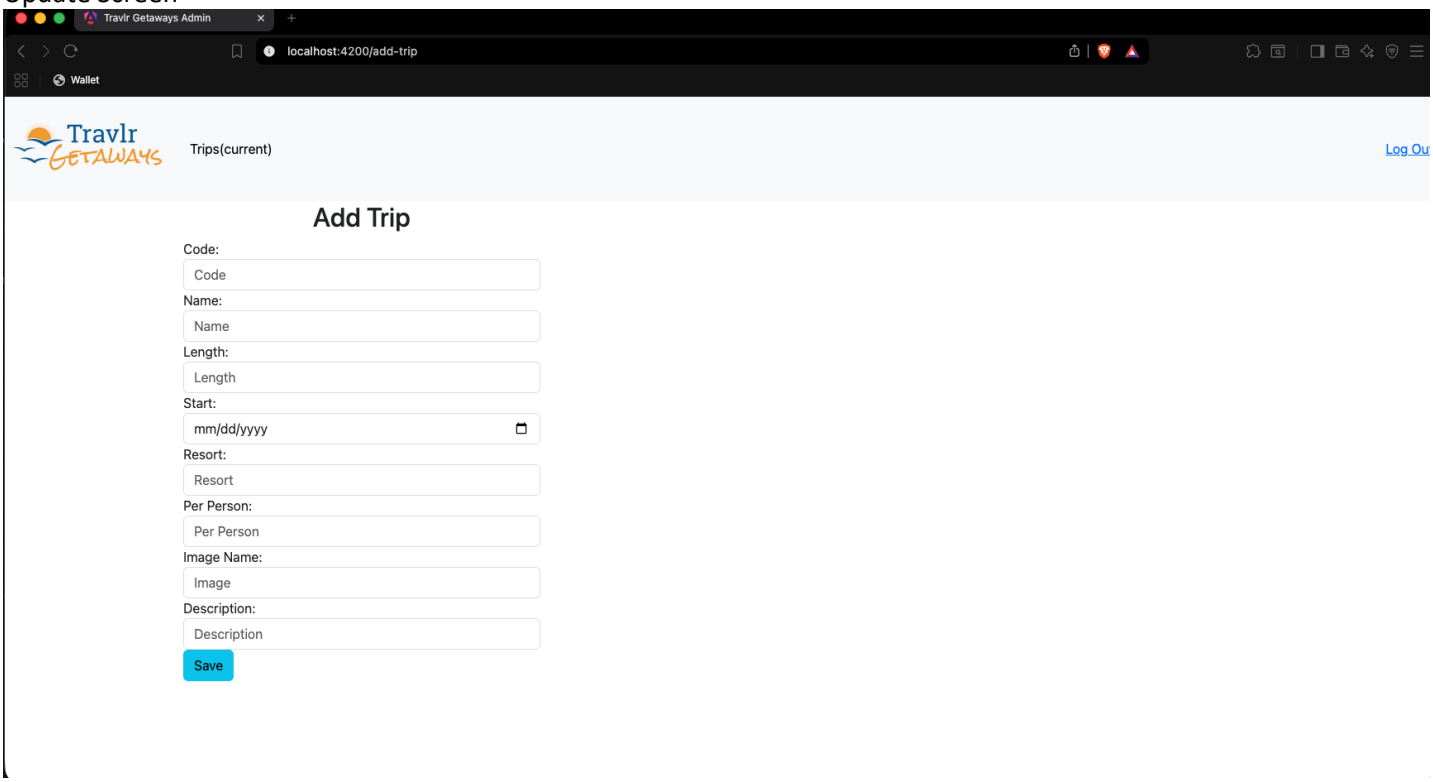
Unique Trip added by me:



Edit Screen



Update Screen



The admin single page application or SPA used to edit and update trips is built using Angular, while the client-facing side is built using Express. These frameworks take a fundamentally different

approach to how they are structured, the main difference being where the code is run. Angular runs on the user's web browser instead of the server. It uses APIs to connect to the server and obtain the data it needs. The Angular project is very modular, structured around three core parts: components, services, and modules, which are used to render the client-side application. A component is the user interface; it controls what the user sees. A service handles the data from the database and other business logic, while a module acts as a container that holds related components and services together. In our project directory, the entire Angular application is stored in the app_admin folder. We created 4 components and 2 services.

The Express application differs from the Angular application in many ways. It is run on the server instead of the user's browser, and it does not strictly require a specific project structure like Angular. It relies on HTTP requests made via routes, processes them with controllers, and returns a view to the user. All the code lives on the server, and the only code sent to the user is in the public folder, often HTML, CSS, and some JavaScript. The Angular SPA offers tremendous functionality compared to the Express application, which is mostly static. The user visits a route, and an HTML file is sent to them to view. While in the Angular SPA, because of the components and services, it is much more dynamic. We were able to create a login form, the ability to add/edit trips and so on.

To verify the Angular SPA is able to get the data it needs, we thoroughly tested the APIs. Without them, the SPA has no way of talking to our database. Using an application called Postman, we can test the APIs in isolation, and with an application called DBeaver, we can view changes to our database easily. See the API endpoint section for a full list of API endpoints and an explanation of JOSN web tokens as well as what the GET, POST, PUT requests are.

Testing the GET endpoints "/api/trips" and "/api/trips/tripcode" was straightforward, as all they do is return one or many trips and require no authentication. Testing the POST "/api/trips" endpoint required us to use a JSON web token we created, which tells the application the user is logged in. This POST method will send JSON to the database and add a new trip. It is used for the add trip functionality in the SPA. The endpoint "/api/trips/:tripcode" is used to edit a trip and requires a JSON web token. We tested it using Postman and the edit trip functionality in the SPA. Lastly, the two endpoints "/api/register" and "/api/login" are what allow the user to create a new account and login back in to get a JSON web token. These endpoints were tested using Postman as well as the login and signout features in the SPA. Much of the logic between them is shared, so the user will need their name to login. This can be changed in the future. DBeaver was used to monitor all changes made to the database for integrity.